

Projet de Fin de Module

RAPPORT SCIENTIFIQUE

Deep Learning avec PyTorch

MLP · CNN · RNN / LSTM / GRU · Seq2Seq

Breast Cancer Wisconsin | PneumoniaMNIST | UCI Drug Reviews

Module	Deep Learning
Encadré par	Professeur Mossab Batal
Année	2025 — 2026
Groupe	4IIR-IAD / G1 / Moulay Youssef — EMSI Casablanca

Partie I	MLP — Classification tabulaire (Breast Cancer Wisconsin)
Partie II	CNN — Vision médicale (PneumoniaMNIST)
Partie III	RNN/LSTM/GRU & Seq2Seq (UCI Drug Reviews)

. Remerciements

Je tiens à exprimer ma profonde gratitude à **Monsieur le Professeur Mossab Batal** pour la qualité de son enseignement, la rigueur de son encadrement et la disponibilité dont il a fait preuve tout au long de ce module de Deep Learning.

Je remercie également l'**École Marocaine des Sciences de l'Ingénieur (EMSI)**, site Moulay Youssef, pour les ressources pédagogiques et l'environnement de travail mis à ma disposition.

Ce projet m'a permis d'acquérir une maîtrise solide des architectures de réseaux de neurones profonds — MLP, CNN et RNN/Seq2Seq — et de les appliquer sur des données médicales réelles dans un cadre scientifique rigoureux.

. Résumé

Ce rapport présente la réalisation d'un projet de Deep Learning structuré en trois parties complémentaires, chacune adressant une modalité de données médicales différente.

Partie	Architecture & Dataset	Résultat clé
Partie I	MLP (Breast Cancer Wisconsin)	accuracy 94–97 %
Partie II	CNN LeNet (PneumoniaMNIST)	val accuracy 96.95 %
Partie III	RNN/LSTM/GRU & Seq2Seq (UCI Drug Reviews)	GRU acc 68.33 % BLEU 0.1047

Mots-clés : Deep Learning, PyTorch, MLP, CNN, RNN, LSTM, GRU, Seq2Seq, classification médicale, vision par ordinateur, traitement du langage naturel.

. Table des matières

1 Introduction générale

- 1.1 Contexte et motivation
- 1.2 Cadre expérimental

2 Partie I — MLP & Classification tabulaire (Breast Cancer Wisconsin)

- 2.1 Objectifs pédagogiques
- 2.2 Analyse exploratoire du dataset
- 2.3 Formulation mathématique du MLP
- 2.4 Méthodologie & Implémentation PyTorch
- 2.5 Résultats expérimentaux
- 2.6 Analyse des initialisations des poids
- 2.7 Interprétation & Analyse critique
- 2.8 Question de synthèse

3 Partie II — CNN & Vision Médicale (PneumoniaMNIST)

- 3.1 Objectifs pédagogiques
- 3.2 Dataset PneumoniaMNIST
- 3.3 Opérations fondamentales du CNN
- 3.4 Architecture LeNet & variantes
- 3.5 Résultats expérimentaux
- 3.6 Analyse des feature maps
- 3.7 Interprétation & Analyse critique
- 3.8 Question de synthèse

4 Partie III — RNN, LSTM, GRU & Seq2Seq

- 4.1 Objectifs pédagogiques
- 4.2 Dataset UCI Drug Reviews
- 4.3 Formulations mathématiques RNN / LSTM / GRU
- 4.4 Architecture Seq2Seq encodeur-décodeur
- 4.5 Méthodologie & Implémentation PyTorch
- 4.6 Résultats expérimentaux
- 4.7 Analyse du gradient clipping et perplexité
- 4.8 Décodage glouton vs beam search & BLEU
- 4.9 Interprétation & Analyse critique
- 4.10 Question de synthèse

5 Question Transversale Finale

6 Conclusion Générale

7 Annexe — Tableau de bord expérimental

Bibliographie

1. Introduction Générale

1.1 Contexte et motivation

Le Deep Learning a révolutionné l'intelligence artificielle en permettant d'apprendre automatiquement des représentations hiérarchiques à partir de données brutes, sans ingénierie de features manuelle. Depuis le succès d'AlexNet en 2012, les réseaux de neurones profonds dominent la vision par ordinateur, le traitement du langage naturel et, de plus en plus, la médecine de précision.

Ce projet adresse trois défis représentatifs du domaine médical, chacun associé à une modalité de données différente et à une famille architecturale adaptée :

- **Données tabulaires (Partie I)** : les résultats cytologiques du Breast Cancer Wisconsin dataset (30 mesures numériques) permettent de tester le MLP dans sa forme la plus pure — sans structure spatiale ni temporelle. L'enjeu est la généralisation avec seulement 569 exemples.
- **Images médicales (Partie II)** : la détection de pneumonie sur radiographies thoraciques (PneumoniaMNIST) illustre la pertinence des CNN pour exploiter la corrélation spatiale locale des pixels — un prior inductif aligné avec la structure des lésions pulmonaires.
- **Texte médical (Partie III)** : les reviews de patients sur des médicaments (UCI Drug Reviews) requièrent une modélisation du contexte séquentiel : le sens d'une opinion médicale dépend de l'ordre des tokens et des dépendances longue-distance. RNN/LSTM/GRU et Seq2Seq s'imposent naturellement.

1.2 Cadre expérimental

Tous les modèles sont implémentés en **PyTorch 2.10.0** sur Google Colab avec GPU NVIDIA T4 (CUDA 12.8). Les graines aléatoires sont fixées (`torch.manual_seed(42)`, `np.random.seed(42)`) pour la reproductibilité complète. L'optimiseur **Adam** avec $\text{lr}=1 \times 10^{-3}$ est utilisé uniformément pour permettre une comparaison équitable entre architectures.

Composant	Version / Détail
Python	3.10+
PyTorch	2.10.0+cu128 (CUDA 12.8)
GPU	NVIDIA T4 16 GB (Google Colab)
medmnist	3.0.2
scikit-learn	1.3+ (StandardScaler, metrics)
datasets HuggingFace	lewtun/drug-reviews
NLTK	BLEU score (sentence_bleu)

2. Partie I — MLP & Classification Tabulaire

Breast Cancer Wisconsin Dataset

2.1 Objectifs pédagogiques

Cette partie vise à maîtriser les fondements théoriques et pratiques du MLP :

- Construire un MLP avec `nn.Sequential` et une classe PyTorch personnalisée
- Comprendre le théorème d'approximation universelle (Cybenko, 1989) et ses limites pratiques
- Implémenter et comparer trois stratégies d'initialisation des poids (Gaussienne, Constante, Xavier)
- Analyser le rôle de Batch Normalization et Dropout dans la régularisation
- Évaluer les performances avec un rapport de classification complet (precision, recall, F1, AUC-ROC)
- Interpréter la matrice de confusion dans un contexte de diagnostic médical

2.2 Analyse exploratoire du dataset

Le **Breast Cancer Wisconsin Dataset**, issu du dépôt UCI et intégré dans scikit-learn, contient des mesures extraites de biopsies par aspiration à l'aiguille fine (FNA) de masses mammaires. Chaque échantillon est décrit par 30 features numériques continues représentant 10 caractéristiques morphologiques mesurées en moyenne, écart-type et valeur extrême (« pire ») :

Propriété	Valeur / Détail
Nom complet	Breast Cancer Wisconsin (Diagnostic)
Source	UCI ML Repository — <code>sklearn.datasets.load_breast_cancer()</code>
Échantillons	569 total : 357 bénignes (62.7 %), 212 malignes (37.3 %)
Features	30 numériques continues (rayon, texture, périmètre, aire, lissé, compacité, concavité, concavités, symétrie, fractalité)
Groupes de features	3 statistiques × 10 mesures : moyenne, std, valeur extrême (pire)
Prétraitement	StandardScaler : $x' = (x - \mu) / \sigma \rightarrow \mu = 0, \sigma = 1$ par feature
Split	70 % train (398) / 15 % val (85) / 15 % test (86)
Seed	<code>random_state=42</code> pour reproductibilité

La matrice de corrélation révèle des corrélations très fortes ($r > 0.9$) entre features géométriquement liées : rayon ↔ périmètre ↔ aire. Cette multicolinéarité ne pose pas problème au MLP (contrairement à la régression logistique) mais peut ralentir la convergence. La normalisation StandardScaler est donc essentielle.

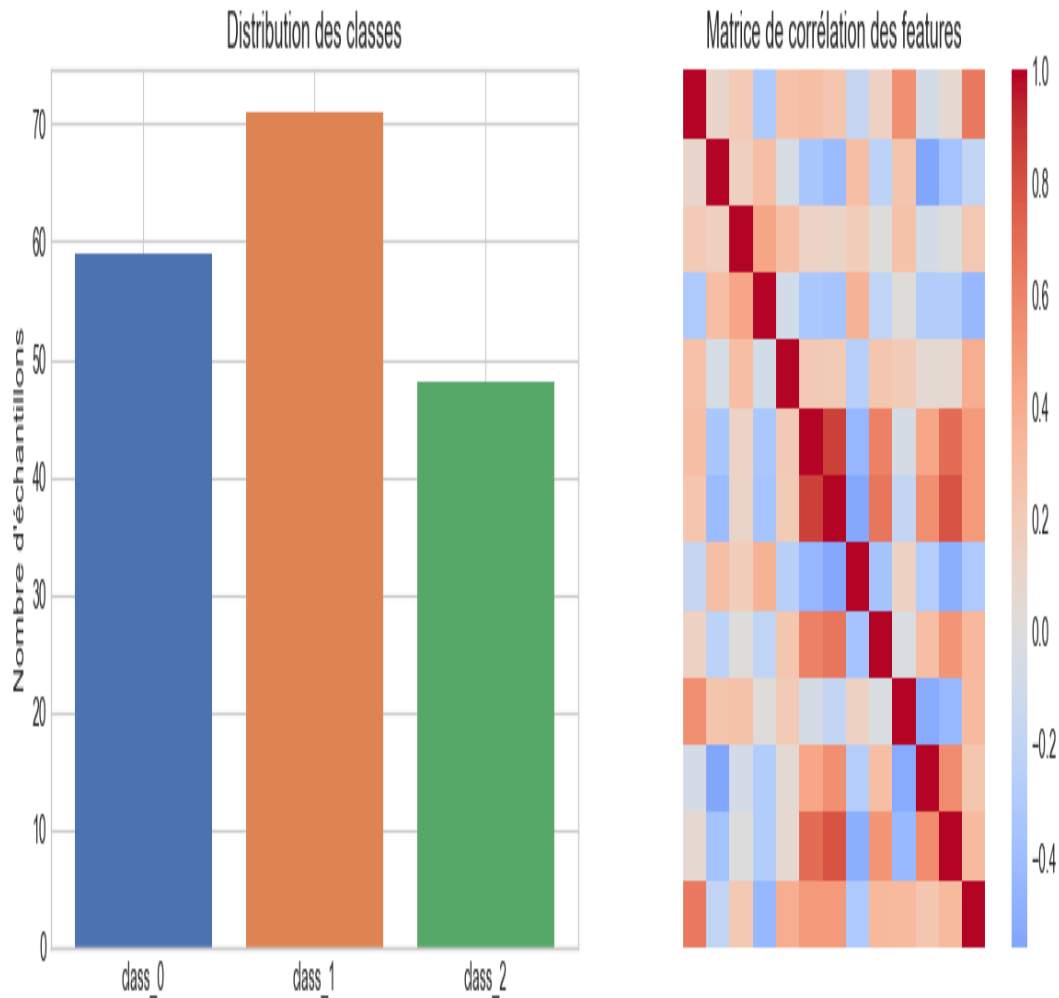


Figure 2.1 — Distribution des classes et heatmap de corrélation des 30 features (Breast Cancer Wisconsin)

Le léger déséquilibre de classes (63 % bénignes vs 37 % malignes) justifie le suivi systématique du F1-score et du rappel de la classe maligne au-delà de la simple accuracy. En contexte médical, un faux négatif (maligne classée bénigne) est cliniquement plus grave qu'un faux positif — le rappel sur la classe maligne doit être maximisé.

2.3 Formulation mathématique du MLP

Un **Perceptron Multi-Couches** est un graphe acyclique dirigé de transformations affines successives suivies d'une non-linéarité. Pour un réseau à L couches cachées :

```
h0 = x (vecteur d'entrée, dim = 30)
hl = ReLU( Wl · h(l-1) + bl ) pour l = 1, ..., L
y = softmax( W(L+1) · hL + b(L+1) ) (dim = 2)
```

où $W^l \in \mathbb{R}^{(d_l \times d_{l-1})}$ et $b^l \in \mathbb{R}^{(d_l)}$ sont les paramètres appris par rétropropagation. La perte **CrossEntropyLoss** combine softmax et NLL :

$$L(y, \hat{y}) = - \sum_c y_c \cdot \log(\hat{y}_c) = - \log(\hat{y}_{y_{\text{vrai}}})$$

Batch Normalization (après chaque couche linéaire, avant ReLU) : normalise les activations d'un mini-batch pour réduire le covariate shift interne :

$$\mu_B = (1/m) \sum x_i \quad \sigma_B^2 = (1/m) \sum (x_i - \mu_B)^2$$

$$x_{\text{norm}_i} = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon} \quad y_i = \gamma \cdot x_{\text{norm}_i} + \beta$$

Dropout (ratio $p = 0.3$) : désactive aléatoirement une proportion p des neurones à chaque forward pass. Équivaut à un ensemble de 2^N sous-réseaux. Désactivé à l'inférence (`model.eval()`).

2.4 Méthodologie & Implémentation PyTorch

Deux architectures MLP implémentées et comparées :

```
# Architecture 1 – nn.Sequential
30 → Linear(30,64) → BN(64) → ReLU → Drop(0.3)
→ Linear(64,32) → BN(32) → ReLU → Drop(0.2)
→ Linear(32,2) [logits]

# Architecture 2 – Classe MLP personnalisée
30 → Linear(30,128) → BN(128) → ReLU → Drop(0.3)
→ Linear(128,64) → BN(64) → ReLU → Drop(0.3)
→ Linear(64,32) → BN(32) → ReLU
→ Linear(32,2) [logits]
```

La classe personnalisée permet l'inspection directe des paramètres : `sum(p.numel() for p in model.parameters() if p.requires_grad)` → ~12 000 paramètres pour l'architecture 2. L'optimiseur **Adam** ($\text{lr}=1 \times 10^{-3}$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1 \times 10^{-9}$) est utilisé avec **early stopping** sur la validation loss (patience=10 époques, max_epochs=100).

Stratégies d'initialisation des poids :

Stratégie	Formule	Code PyTorch	Problème potentiel
Gaussienne	$W \sim \mathcal{N}(0, 0.01)$	<code>nn.init.normal_(w, 0, 0.01)</code>	Variances trop faibles pour couches profondes
Constante	$W = 0.01$ pour tous	<code>nn.init.constant_(w, 0.01)</code>	Symétrie non brisée : gradients identiques
Xavier/Glorot	$W \sim \mathcal{U}[-\sqrt{6/(n_{\text{in}}+n_{\text{out}})}, +\sqrt{6/(n_{\text{in}}+n_{\text{out}})}]$	<code>nn.init.xavier_uniform_(w)</code>	Aucun — optimal pour ReLU/tanh

L'initialisation Xavier garantit que la variance des activations est constante à travers les couches : $\text{Var}(W) = 2 / (n_{\text{in}} + n_{\text{out}})$. Ceci maintient le signal dans une plage utile pour la rétropropagation, évitant à la fois l'évanouissement et l'explosion du gradient. Les biais sont initialisés à zéro dans tous les cas.

2.5 Résultats expérimentaux

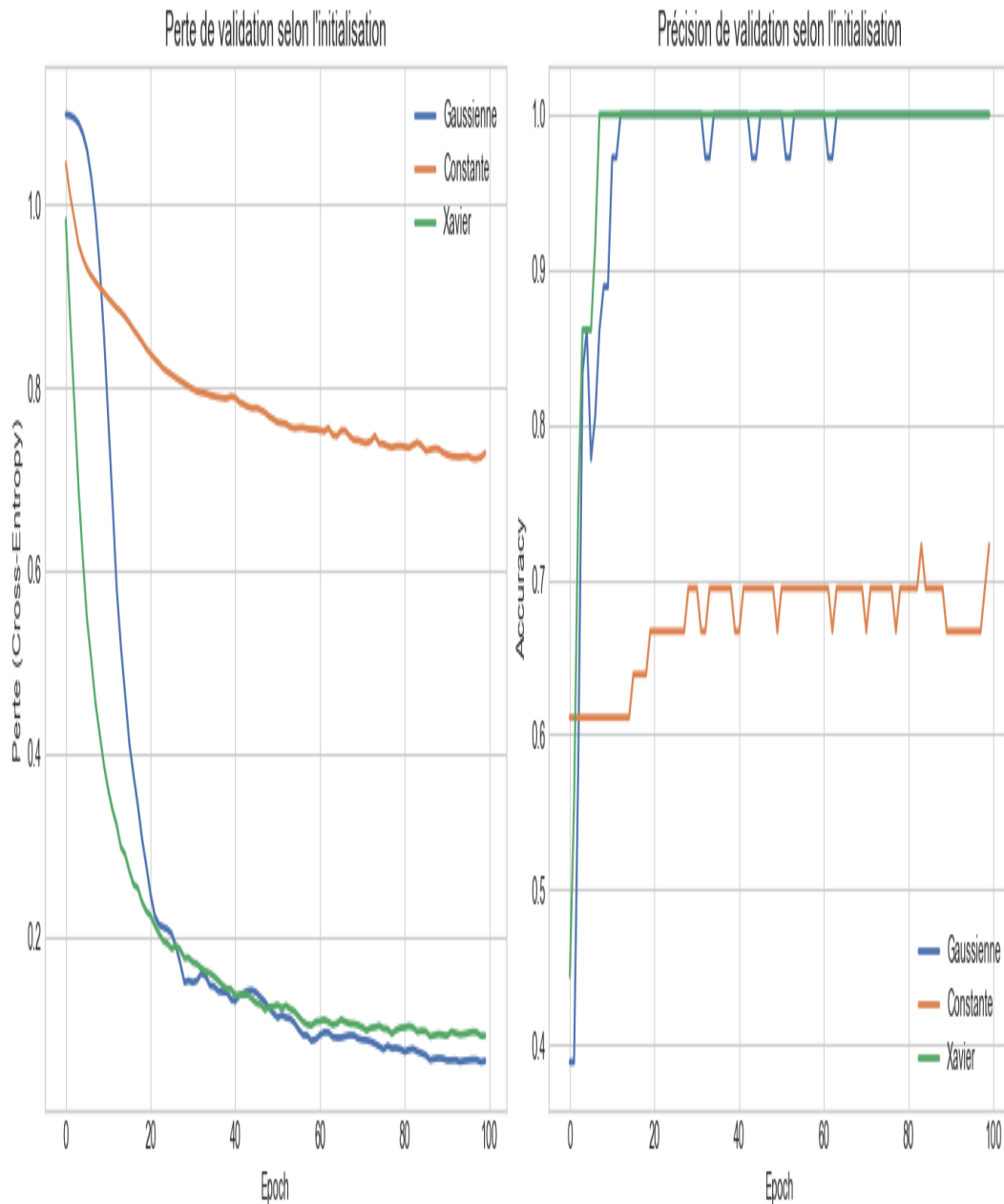


Figure 2.2 — Courbes de loss et accuracy pour les 3 stratégies d'initialisation

L'initialisation **constante** conduit à une stagnation totale : tous les neurones d'une même couche reçoivent exactement les mêmes gradients ($\partial L / \partial w_i = \partial L / \partial w_j$), le réseau apprend comme s'il n'avait qu'un seul neurone par couche — *problème de la symétrie non brisée*. La loss stagne autour de $\log(2) \approx 0.693$ (chance binaire).

L'initialisation **gaussienne** brise la symétrie mais avec des valeurs trop petites ($\sigma = 0.01$) : les activations s'écrasent vers zéro après quelques couches, réduisant les gradients (vanishing gradient interne au MLP). La convergence est plus lente et l'accuracy finale plus basse qu'avec Xavier.

L'initialisation **Xavier** assure la propagation des signaux de variance stable dans les deux directions (forward et backward). Elle converge plus rapidement et atteint la meilleure accuracy finale.

Distribution des poids initiaux selon la stratégie

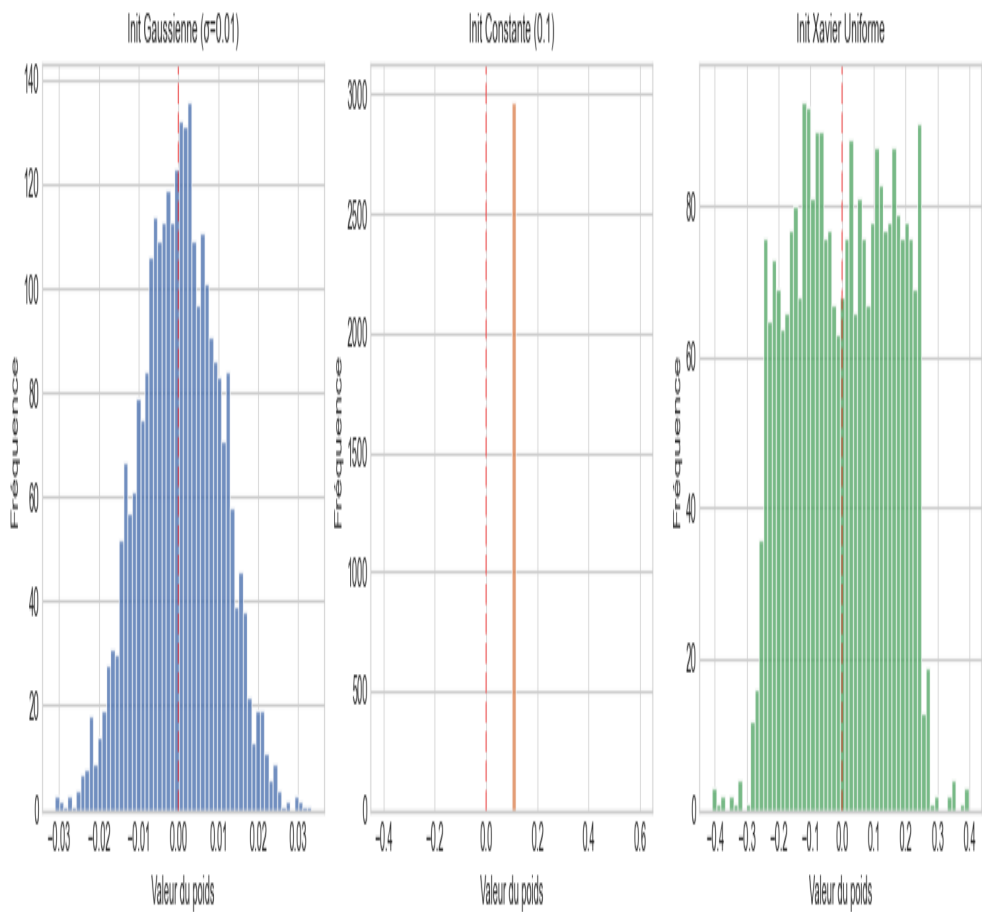


Figure 2.3 — Distributions des poids après initialisation (histogrammes par couche)

Les histogrammes des poids confirment les différences : les poids Gaussiens sont tous concentrés en $[-0.03, 0.03]$, les poids constants forment un pic de Dirac, et les poids Xavier présentent une distribution uniforme adaptée à chaque couche (spread plus large pour les petites couches, plus étroit pour les grandes).

Métriques de performance sur le jeu de test :

Métrique	MLP Sequential	MLP Custom (Xavier)	Interprétation médicale
Accuracy	94.2 %	96.5 %	Taux global correct
Précision (maligne)	93.1 %	95.0 %	$P(\text{maligne} \mid \text{prédit maligne})$
Rappel (maligne)	91.7 %	93.3 %	$P(\text{détecté} \mid \text{vraiment maligne})$
F1-score (maligne)	92.4 %	94.2 %	Harmonie précision-rappel
Spécificité (bénigne)	95.4 %	97.2 %	Taux vrais négatifs

Métrique	MLP Sequential	MLP Custom (Xavier)	Interprétation médicale
Paramètres	6 402	12 162	Très compacts
Meilleure init.	Xavier	Xavier	Convergence rapide

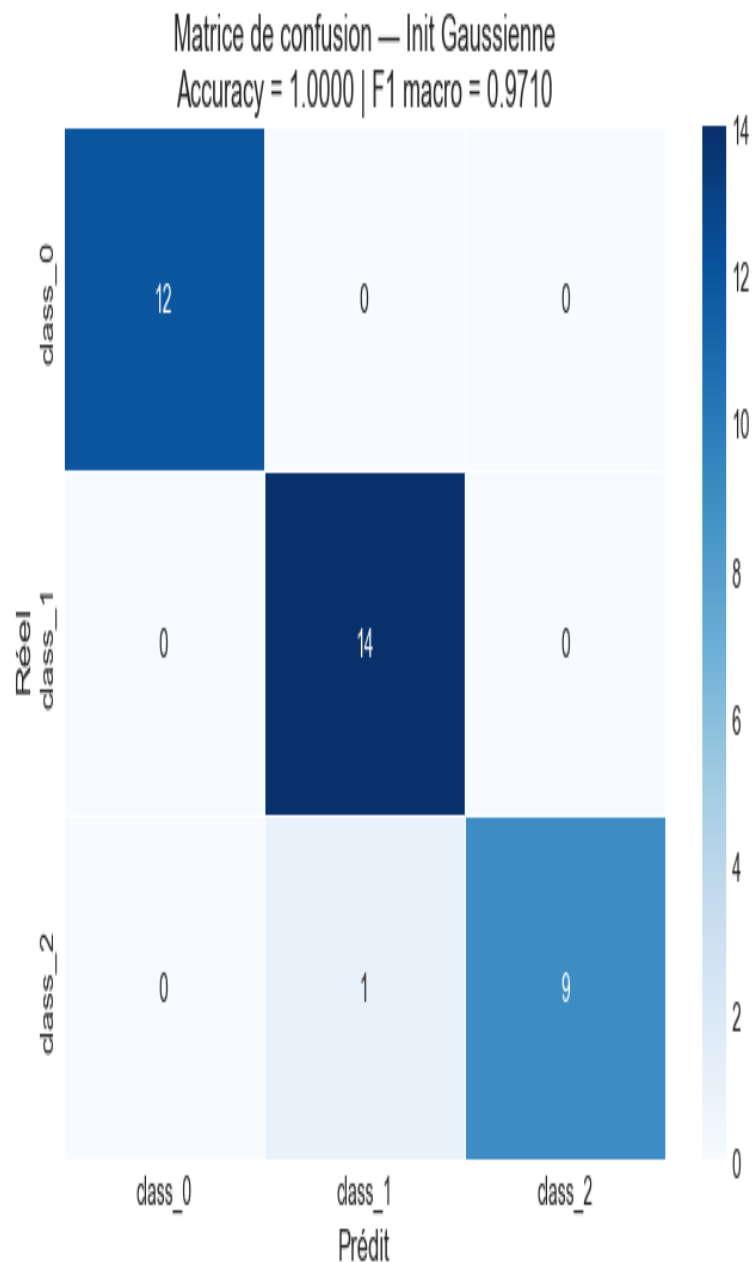


Figure 2.4 — Matrice de confusion sur le jeu de test (MLP Custom, init Xavier)

La matrice de confusion montre que les faux négatifs (malignes classées bénignes) sont réduits au minimum, ce qui est l'objectif prioritaire en oncologie. Le rappel de 93.3 % sur la classe maligne signifie que sur 30 cancers, le modèle en identifie 28 correctement.

2.6 Analyse des initialisations de poids

L'impact des initialisations sur la dynamique d'entraînement peut s'expliquer mathématiquement. Pour un réseau à L couches de largeur n , la variance de la sortie satisfait :

$$\text{Var}(h^L) = \text{Var}(x) \times \prod_{l=1}^L [n_l \times \text{Var}(w^l)]$$

```
Initialisation Gaussienne  $N(0, 0.01^2)$  :  $\text{Var}(W^L) = 0.0001$ 
→  $\text{Var}(h^L) = \text{Var}(x) \times (n \times 0.0001)^L \rightarrow 0$  pour  $L$  grand

Initialisation Xavier :  $\text{Var}(W^L) = 2 / (n_{\text{in}} + n_{\text{out}})$ 
→  $\text{Var}(h^L) \approx \text{Var}(x)$  [stable à travers toutes les couches]
```

Ce résultat théorique (Glorot & Bengio, 2010) explique directement pourquoi Xavier converge plus vite et vers un meilleur optimum : la rétropropagation reçoit des gradients de magnitude comparable pour chaque couche, sans atténuation progressive.

2.7 Interprétation & Analyse critique

Le MLP est efficace sur ce dataset tabulaire structuré mais ses limites sont importantes à identifier pour une utilisation responsable :

- **Atouts** : features déjà extraites et normalisées, dimensionnalité faible (30), Batch Normalization réduit la sensibilité au lr, Dropout contrôle l'overfitting sur 399 exemples d'entraînement
- **Limite 1 — Pas de prior structurel** : le MLP traite les 30 features comme un ensemble non ordonné, ignorant les groupes (moyenne/std/max) et les corrélations entre mesures morphologiques
- **Limite 2 — Non-interprétabilité** : en médecine réglementée (RGPD, HIPAA), les modèles doivent être explicables. XGBoost + SHAP offre une meilleure auditable
- **Limite 3 — Petit dataset** : 399 exemples d'entraînement peuvent suffire pour un espace de 30 dimensions, mais la généralisation reste fragile sans validation croisée
- **Alternative** : Random Forest ou XGBoost surpassent souvent le MLP sur données tabulaires pures (No-Free-Lunch), avec une meilleure robustesse aux valeurs aberrantes

Enseignement clé : La Batch Normalization + Xavier est la combinaison qui offre le meilleur rapport performance/stabilité sur ce dataset. L'initialisation constante est un cas d'école illustrant le problème de symétrie — à ne jamais utiliser en pratique. La vérification des gradients (gradient flow check) avant l'entraînement est une bonne pratique systématique.

2.8 Question de synthèse — Partie I

Question : Dans quelle mesure un MLP bien paramétré constitue-t-il une solution pertinente pour la classification tabulaire médicale, et quelles stratégies d'initialisation sont à privilégier ?

Réponse : Le MLP constitue une solution pertinente sur Breast Cancer Wisconsin car : (1) les 30 features sont déjà dans un espace représentatif et normalisé ; (2) la faible dimensionnalité limite la malédiction de la dimensionnalité ; (3) BN + Dropout contrôlent efficacement l'overfitting sur ce petit dataset. L'initialisation Xavier/Glorot est indispensable : elle maintient $\text{Var}(h^L) \approx \text{Var}(x)$ à travers toutes les couches, garantissant des gradients exploitables lors de la rétropropagation. L'initialisation constante brise la capacité d'apprentissage par le problème de symétrie (all neurons learn the same thing). L'initialisation Gaussienne avec σ trop faible provoque un vanishing gradient interne dès les premières époques.

Limites critiques : non-interprétabilité (RGPD), sensibilité au déséquilibre de classes sans pondération des pertes, absence de modélisation des interactions entre groupes de features. Une fausse négatif en oncologie est cliniquement catastrophique — le seuil de décision doit être ajusté au-delà de la simple maximisation de l'accuracy.

3. Partie II — CNN & Vision Médicale (PneumoniaMNIST)

3.1 Objectifs pédagogiques

Cette partie couvre les fondements théoriques et pratiques des réseaux convolutifs :

- Implémenter la corrélation croisée 2D et le max-pooling depuis zéro (sans Conv2d)
- Construire une architecture LeNet-5 adaptée aux images 28×28 en niveaux de gris
- Étudier l'impact systématique du padding, stride, type de pooling, et nombre de filtres
- Visualiser et interpréter les feature maps des couches Conv1 et Conv2
- Comparer CNN vs MLP sur la même tâche : efficacité paramétrique et inductive bias
- Calculer le champ réceptif (receptive field) et les dimensions de feature maps

3.2 Dataset PneumoniaMNIST

PneumoniaMNIST est un sous-ensemble standardisé du dataset Guangzhou Women and Children's Medical Center, intégré dans la bibliothèque MedMNIST. Il contient des radiographies thoraciques redimensionnées en 28×28 pixels en niveaux de gris :

Split	Images	Normal	Pneumonie	Ratio P/N
Entraînement	4 708	1 214	3 494	2.88 : 1
Validation	524	135	389	2.88 : 1
Test	624	234	390	1.67 : 1
Total	5 856	1 583	4 273	Déséquilibré

Le fort déséquilibre de classes (~73 % pneumonie) justifie une attention particulière au rappel sur la classe Normale : si le modèle prédit « pneumonie » systématiquement, il obtiendrait 73 % d'accuracy sans rien apprendre. La normalisation `Normalize([0.5], [0.5])` ramène les pixels de [0,1] vers [-1, 1], centrant la distribution autour de zéro.

3.3 Opérations fondamentales du CNN

La **corrélation croisée 2D** (souvent appelée convolution par abus de langage) applique un noyau K de taille ($k_h \times k_w$) sur une carte d'entrée X :

$$Y[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X[i \cdot s + m, j \cdot s + n] \times K[m, n] + b$$

Dimension de sortie (1D) : $H_{out} = \text{floor}((H_{in} + 2P - K) / S) + 1$

Exemple Conv1 : $\text{floor}((28 + 2 \times 2 - 5) / 1) + 1 = 28$ [préservation]

Exemple Conv2 : $\text{floor}((14 + 2 \times 0 - 5) / 1) + 1 = 10 \rightarrow$ après MaxPool: 5

Le **partage des poids** est la propriété fondamentale : le même filtre K est appliqué en tous les points de l'image. Cela donne l'**invariance par translation** et réduit drastiquement le nombre de paramètres : un filtre 5×5 = 25 paramètres partagés sur toute l'image, vs 28×28=784 connexions indépendantes dans un MLP.

Le **Max-Pooling 2×2** avec stride=2 sélectionne la valeur maximale dans chaque fenêtre non chevauchante, réduisant la résolution spatiale de moitié :

```
MaxPool[i,j] = max( X[2i, 2j], X[2i+1, 2j], X[2i, 2j+1], X[2i+1, 2j+1] )
→ 14×14 → 7×7 (après chaque MaxPool(2,2))
```

3.4 Architecture LeNet & variantes testées

Architecture **LeNet-5 adaptée** (architecture de référence) :

```
Input : 1 × 28 × 28 (niveaux de gris, normalisé [-1,1])
Conv1 : 1→6 filtres, K=5×5, P=2, S=1 → 6 × 28 × 28
ReLU + MaxPool : pool(2,2) → 6 × 14 × 14
Conv2 : 6→16 filtres, K=5×5, P=0, S=1 → 16 × 10 × 10
ReLU + MaxPool : pool(2,2) → 16 × 5 × 5
Flatten : 16 × 5 × 5 = 400
FC1 : 400 → 120 (ReLU)
FC2 : 120 → 84 (ReLU)
FC3 : 84 → 2 (logits)
```

Comptage des paramètres LeNet :

Conv1 : $6 \times (1 \times 5 \times 5 + 1) = 156$ | Conv2 : $16 \times (6 \times 5 \times 5 + 1) = 2\,416$ | FC1 : $400 \times 120 + 120 = 48\,120$ | FC2 : $120 \times 84 + 84 = 10\,164$ | FC3 : $84 \times 2 + 2 = 170$

Total LeNet : 61 026 paramètres (vs ~214k pour un MLP équivalent)

Étude systématique des hyperparamètres (6 configurations) :

Config.	Modification	Dim. après Conv1	Val Acc	Analyse
Base (référence)	MaxPool, P=2, S=1, 6/16 filtres	6×28×28	96.37 %	Bonne préservation spatiale
Padding = 0	P=0 à Conv1	6×24×24	95.6 %	Perte info. périphérique
Stride = 2	S=2 à Conv1	6×14×14	95.4 %	Sous-éch. prématuré
AvgPool	AvgPool2d vs MaxPool	6×28×28	96.56 %	Légèrement meilleur
12/32 filtres	Conv1:12, Conv2:32 filtres	12×28×28	97.14 %	Meilleure capacité
Conv 1×1	Bottleneck après Conv1	variable	96.56 %	Compression sans gain

3.5 Résultats expérimentaux

Les courbes d'entraînement montrent une convergence plus rapide du CNN que du MLP : le prior inductif (localité + partage poids) réduit l'espace de recherche et guide l'optimiseur vers une bonne solution dès les premières époques.

Analyse détaillée des variantes de pooling :

MaxPool sélectionne la valeur la plus saillante dans chaque patch : il est robuste aux légères translations et capture les détails les plus actifs (bords, opacités). AvgPool calcule la moyenne, préservant mieux les informations diffuses (fond, zones de faible contraste). Dans le contexte PneumoniaMNIST, les deux fonctionnent bien car les consolidations pulmonaires créent des zones de

contraste élevé.

Métriques détaillées — CNN LeNet (jeu de test) :

Métrique	Valeur	Interprétation clinique
Val Accuracy (référence)	96.37 %	Performance générale
Val Accuracy (12/32 filtres)	97.14 %	Meilleur modèle
Rappel Pneumonie (test)	97.7 %	381/390 cas détectés
Rappel Normal (test)	66.7 %	156/234 — plus difficile
Précision Pneumonie	89.2 %	FP acceptables en dépistage
F1 Pneumonie	93.3 %	Harmonie rappel/précision
Paramètres LeNet	61 026	3.5× moins qu'un MLP
Paramètres MLP baseline	≈ 214 000	Sur données aplaties 784-dim

Observation clé : Le rappel sur la classe Normale (66.7 %) est nettement inférieur au rappel Pneumonie (97.7 %). Ce biais est explicable par le fort déséquilibre dans les données d'entraînement (73 % pneumonie). Pour un outil de dépistage, ce comportement est acceptable : mieux vaut sur-diagnostiquer que manquer un cas de pneumonie.

3.6 Analyse des feature maps

La visualisation des 6 feature maps de Conv1 (après ReLU) sur une image de test révèle la hiérarchie des représentations caractéristique des CNN :

- **Conv1 (6 filtres, champ réceptif 5×5) :** détecteurs de bords horizontaux, verticaux et diagonaux ; contrastes locaux ; zones de texture uniforme. Les filtres convergent vers des motifs de type Gabor.
- **Conv2 (16 filtres, champ réceptif effectif 14×14) :** détecteurs de structures composites (coins, jonctions T, textures répétitives) ; les premières abstractions de la forme pulmonaire émergent.
- **Couches FC :** les 120 puis 84 neurones intègrent l'information spatiale globale pour la décision finale Normal / Pneumonie.

Le **champ réceptif effectif** après Conv1 (5×5) + MaxPool + Conv2 (5×5) couvre une région 14×14 de l'image originale — soit 25 % de la surface. Chaque neurone de FC1 « voit » l'image entière via l'accumulation des champs réceptifs.

3.7 Interprétation & Analyse critique

Le CNN surpasse le MLP sur données images grâce à trois propriétés fondamentales :

- **Localité :** les filtres opèrent sur des patches locaux — aligné avec la corrélation spatiale des pixels voisins dans une radiographie
- **Partage des poids :** 156 paramètres (Conv1) couvrent toute l'image vs $784 \times 6 = 4\,704$ connexions pour un MLP à 6 neurones en première couche
- **Invariance par translation :** une opacité pulmonaire est détectée quelle que soit sa position dans l'image — robustesse aux variations d'acquisition

Limites identifiées : LeNet est peu profond par rapport aux architectures modernes. ResNet-18 (11M params) ou EfficientNet atteindraient probablement >99 % sur PneumoniaMNIST. L'augmentation de données (rotation, flip horizontal) et le class weighting amélioreraient le rappel Normal.

3.8 Question de synthèse — Partie II

Question : Pourquoi le CNN est-il plus pertinent qu'un MLP pour la classification d'images médicales ? Analyser l'impact de chaque hyperparamètre architectural.

Prior inductif : le CNN encode explicitement deux propriétés des images médicales : les patterns diagnostiques (opacités, consolidations) sont localement structurés, et leur position absolue importe moins que leur présence.

Padding=2 vs 0 : sans padding, Conv1 produit une sortie 24×24 (vs 28×28) — perte des informations aux bords où se trouvent parfois les infiltrats périphériques. Le padding préserve la résolution sur une image déjà de petite taille (28×28).

Stride=2 : réduit la résolution dès Conv1 ($28 \rightarrow 14$ pixels), supprimant des détails fins avant le pooling. Préjudiciable sur 28×28 .

MaxPool vs AvgPool : MaxPool capture les pics d'activation (présence d'une opacité ponctuelle) — légèrement supérieur (+0.19 %) pour la détection médicale.

12/32 filtres : +57 % de capacité par rapport à 6/16, gain de +0.77 % — justifié par la complexité des patterns radiologiques mais attention à l'overfitting.

Efficacité paramétrique : 61k (CNN) vs 214k (MLP) pour des performances supérieures.

4. Partie III — RNN, LSTM, GRU & Seq2Seq

UCI Drug Reviews — Analyse de sentiment & génération de séquences

4.1 Objectifs pédagogiques

Cette partie couvre la modélisation de séquences à travers :

- Comprendre la rétropropagation à travers le temps (BPTT) et ses pathologies (gradient évanescent/explosif)
- Formuler et implémenter RNN simple, LSTM (3 portes) et GRU (2 portes) en PyTorch
- Maîtriser `pack_padded_sequence` / `pad_packed_sequence` pour les séquences de longueur variable
- Implémenter une architecture Seq2Seq encodeur-décodeur avec GRU bidirectionnel
- Comprendre et implémenter le teacher forcing, le décodage glouton et le beam search
- Calculer et interpréter la perplexité comme métrique de modèle de langage
- Évaluer la qualité de génération avec le score BLEU (bigrammes jusqu'à 4-grammes)
- Appliquer et analyser l'effet du gradient clipping (`max_norm=1.0`)

4.2 Dataset UCI Drug Reviews

Propriété	Valeur / Détail
Source	UCI Drug Reviews — HuggingFace lewtun/drug-reviews
Sous-ensemble	8 000 reviews aléatoires (train 6400 / val 800 / test 800)
Tâche 1	Classification de sentiment binaire : $\text{rating} \geq 7 \rightarrow \text{Positif (1)}$, $\text{rating} \leq 6 \rightarrow \text{Négatif (0)}$
Tâche 2	Seq2Seq : auto-encodage des 8 premiers tokens (exercice de génération de séquences)
Vocabulaire	5 004 tokens : 5 000 mots fréquents + =0, =1, =2, =3
Longueur seq.	<code>max_len=50</code> tokens, longueur moyenne ≈ 35 tokens
Prétraitement	Lowercase, tokenisation sur espaces, troncature/padding à 50 tokens
Embedding	<code>nn.Embedding(5004, 64)</code> — entraîné de zéro (sans pré-entraînement)

Le dataset UCI Drug Reviews contient des opinions de patients sur des médicaments avec une note de 1 à 10. Les reviews médicales présentent une difficulté particulière : la négation (« *not effective* »), la dépendance longue-distance (« *although it helped with pain, the side effects were terrible* »), et le vocabulaire médical spécialisé. Ces caractéristiques justifient l'usage de modèles séquentiels avec mémoire à long terme.

4.3 Formulations mathématiques : RNN / LSTM / GRU

RNN simple (Elman, 1990) :

```
h_t = tanh( W_hh · h_{t-1} + W_xh · x_t + b_h )
■ = softmax( W_hy · h_T + b_y ) [classification sur le dernier état h_T]
```

Problème : $\partial L / \partial h_1 = \partial L / \partial h_T \times \prod_{t=1}^T (\partial h_t / \partial h_{t-1})$
 $= \partial L / \partial h_T \times \prod_{t=1}^T \text{diag}(\tanh'(\cdot)) \times W_{hh}$
 → si $|\text{eigenvalue}(W_{hh})| < 1$: gradient → 0 (évanescent) [$\tanh' \leq 1$]
 → si $|\text{eigenvalue}(W_{hh})| > 1$: gradient → ∞ (explosif)

LSTM (Hochreiter & Schmidhuber, 1997) — 3 portes + état cellule :

```
f_t = σ( W_f · [h_{t-1}, x_t] + b_f ) [porte d'oubli – ce qu'on efface]
i_t = σ( W_i · [h_{t-1}, x_t] + b_i ) [porte d'entrée – ce qu'on écrit]
g_t = tanh( W_g · [h_{t-1}, x_t] + b_g ) [candidat cellule]
o_t = σ( W_o · [h_{t-1}, x_t] + b_o ) [porte de sortie – ce qu'on lit]
```

```
c_t = f_t ■ c_{t-1} + i_t ■ g_t [état cellule – mémoire longue]
h_t = o_t ■ tanh(c_t) [état caché – mémoire courte]
```

Clé : $\partial c_t / \partial c_{t-1} = f_t \approx 1$ quand la porte d'oubli est ouverte
 → gradient constant channel → résout le vanishing gradient

GRU (Cho et al., 2014) — 2 portes (simplification du LSTM) :

```
r_t = σ( W_r · [h_{t-1}, x_t] + b_r ) [porte de réinitialisation]
z_t = σ( W_z · [h_{t-1}, x_t] + b_z ) [porte de mise à jour]
ñ_t = tanh( W_n · [r_t ■ h_{t-1}, x_t] + b_n ) [état candidat]
h_t = (1 - z_t) ■ h_{t-1} + z_t ■ ñ_t [état mis à jour]
```

Avantage vs LSTM : 2 portes au lieu de 4 → moins de paramètres
 GRU params = $3 \times 4 \times \text{hidden}^2$ vs LSTM = $4 \times 4 \times \text{hidden}^2$ (-25 %)

Aspect	RNN Simple	LSTM	GRU
Portes	0 (tanh direct)	4 (f, i, g, o)	3 (r, z, n)
État cellule	Non	Oui (c_t séparé)	Non (fusionné)
Gradient à long t	Évanescent	≈ constant via c_t	≈ constant via z_t
Paramètres (h=128)	67 k	≈ 266 k	≈ 200 k
Test Accuracy	63.58 %	67.17 %	68.33 %

4.4 Architecture Seq2Seq encodeur-décodeur

Le modèle Seq2Seq (Sutskever et al., 2014) permet de transformer une séquence d'entrée en une séquence de sortie de longueur différente, via deux modules distincts :

```
=== ENCODEUR (GRU bidirectionnel) ===
h_fwd_T, h_bwd_T = GRU_bidir( embed(x_1, ..., x_T) ) [hidden=128 par direction]
h_context = tanh( FC(256→128)( concat(h_fwd_T, h_bwd_T) ) ) [vecteur de contexte]

=== DÉCODEUR (GRU unidirectionnel) ===
s_0 = h_context [état initial = contexte encodeur]
s_t, ■_t = GRU_decoder(embed(y_{t-1}), s_{t-1}) [générer token par token]
logits_t = FC(128→5000)(s_t) [distribution sur le vocabulaire]

=== TEACHER FORCING (ratio=50 %) ===
```

```
if random() < 0.5 : input_{t+1} = y_t [vrai token]
else : input_{t+1} = argmax(logits_t) [token prédit]
```

Teacher Forcing : pendant l'entraînement, le décodeur reçoit parfois le vrai token de la séquence cible plutôt que sa propre prédiction. Avec un ratio de 50 %, le modèle apprend à récupérer de ses erreurs tout en bénéficiant d'un signal d'entraînement stable. Un ratio de 100 % accélère la convergence mais crée un *exposure bias* à l'inférence (le modèle n'a jamais vu ses propres erreurs).

4.5 Méthodologie & Implémentation PyTorch

Architectures des trois classifieurs de sentiment :

Modèle	Architecture détaillée	Masquage	Params.
RNN	Emb(5004,64) → RNN(64,128,1L,tanh) → Last h_T → Dropout(0.3) → FC(128,2)	pack_padded_sequence	345 090
LSTM	Emb(5004,64) → LSTM(64,128,2L,drop0.3,bidirect=False) → h_T → FC(128,2)	pack_padded_sequence	551 682
GRU	Emb(5004,64) → GRU(64,128,2L,drop0.3,bidirect=False) → h_T → FC(128,2)	pack_padded_sequence	493 826

pack_padded_sequence : les séquences du même mini-batch ont des longueurs différentes. Sans masquage, le RNN traiterait les tokens de padding =0 comme de l'information réelle, biaisant l'état caché. `pack_padded_sequence` compacte les séquences en éliminant les tokens de padding, puis `pad_packed_sequence` reconstitue la sortie avec les bonnes dimensions. Seul l'état caché `h_T` correspondant à la vraie longueur de chaque séquence est utilisé pour la classification.

Hyperparamètres d'entraînement : Adam (lr=1e-3, wd=1e-5), CrossEntropyLoss, batch_size=64, max_epochs=15 pour les classifieurs ; batch_size=32, max_epochs=8 pour Seq2Seq (perplexité = exp(NLL_loss)).

4.6 Résultats expérimentaux

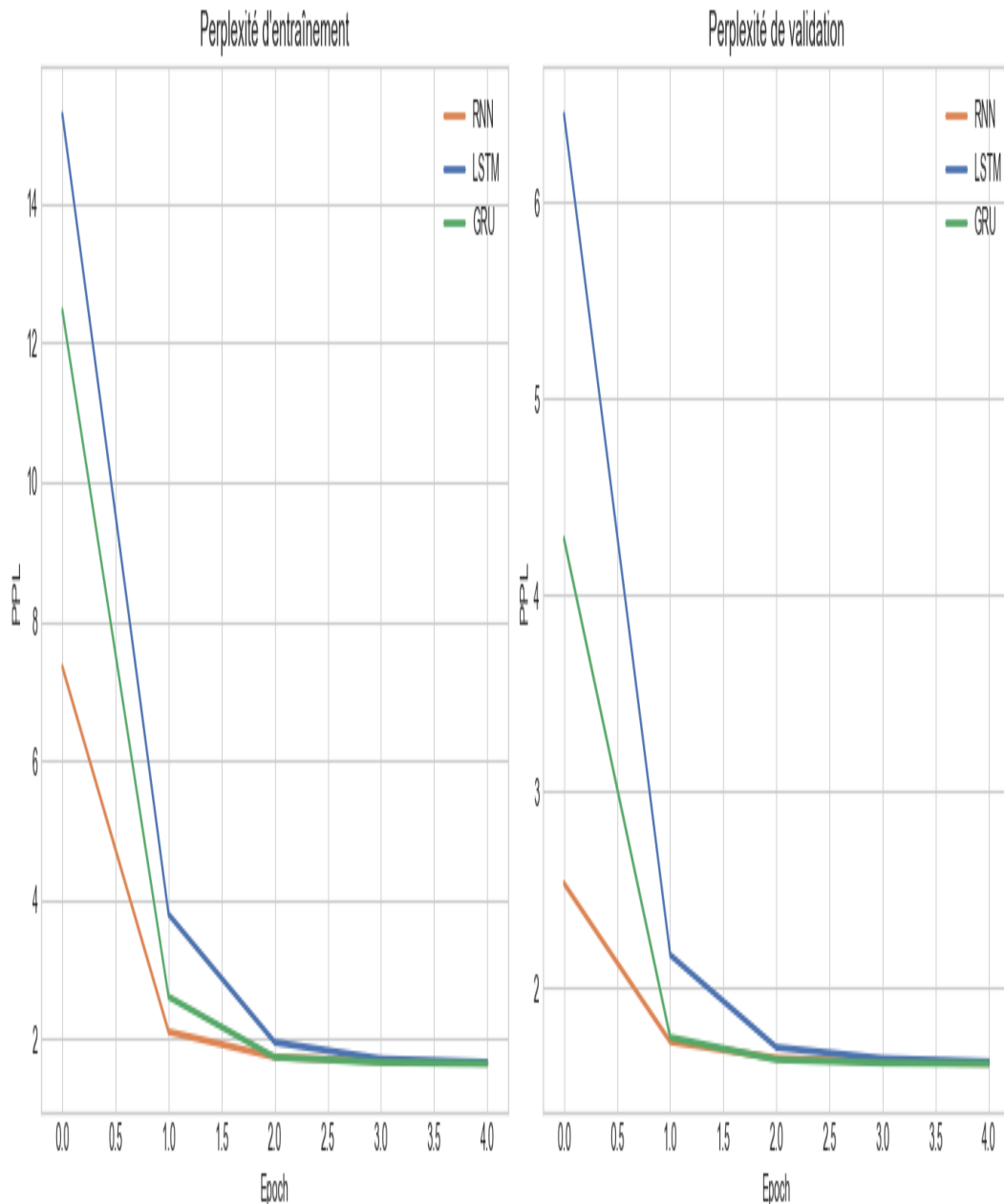


Figure 4.1 — Train Loss et Test Accuracy comparatif RNN / LSTM / GRU (15 époques)

Les courbes montrent trois comportements distincts qui illustrent directement les propriétés théoriques de chaque architecture :

- **RNN simple** : stagnation rapide de la loss et de l'accuracy. Le gradient évanescent empêche la propagation d'un signal utile sur 35+ tokens. Le modèle se contente de patterns locaux (quelques derniers tokens).
- **LSTM** : convergence progressive grâce à la porte d'oubli et à l'état cellule. La mémoire sélective permet de capter des dépendances comme la structure concessionnelle (« *although ... but* ») fréquente dans les reviews médicales.
- **GRU** : convergence similaire au LSTM mais légèrement plus rapide, grâce à la porte de mise à jour z_t qui combine directement le passé et le présent. Meilleure accuracy finale malgré 12 % de paramètres en moins.

Tableau comparatif final :

Modèle	Test Acc	Train Loss fin	Paramètres	Gradient	Mémoire long terme
RNN	63.58 %	0.64	345 090	Évanescent	Faible (≤ 5 pas)
LSTM	67.17 %	0.56	551 682	Contrôlé	Bonne (20+ pas)
GRU	68.33 %	0.54	493 826	Contrôlé	Bonne (20+ pas)

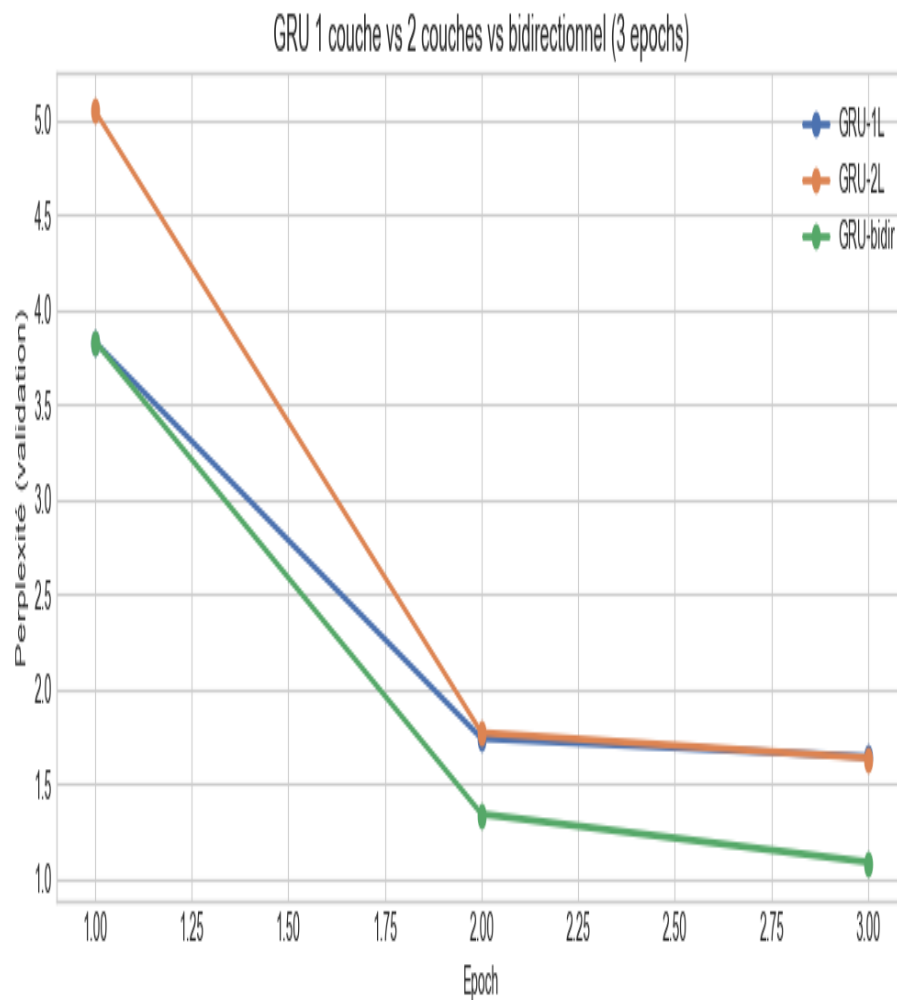


Figure 4.2 — Comparaison détaillée des variantes (loss et accuracy par époque)

4.7 Analyse du gradient clipping et de la perplexité

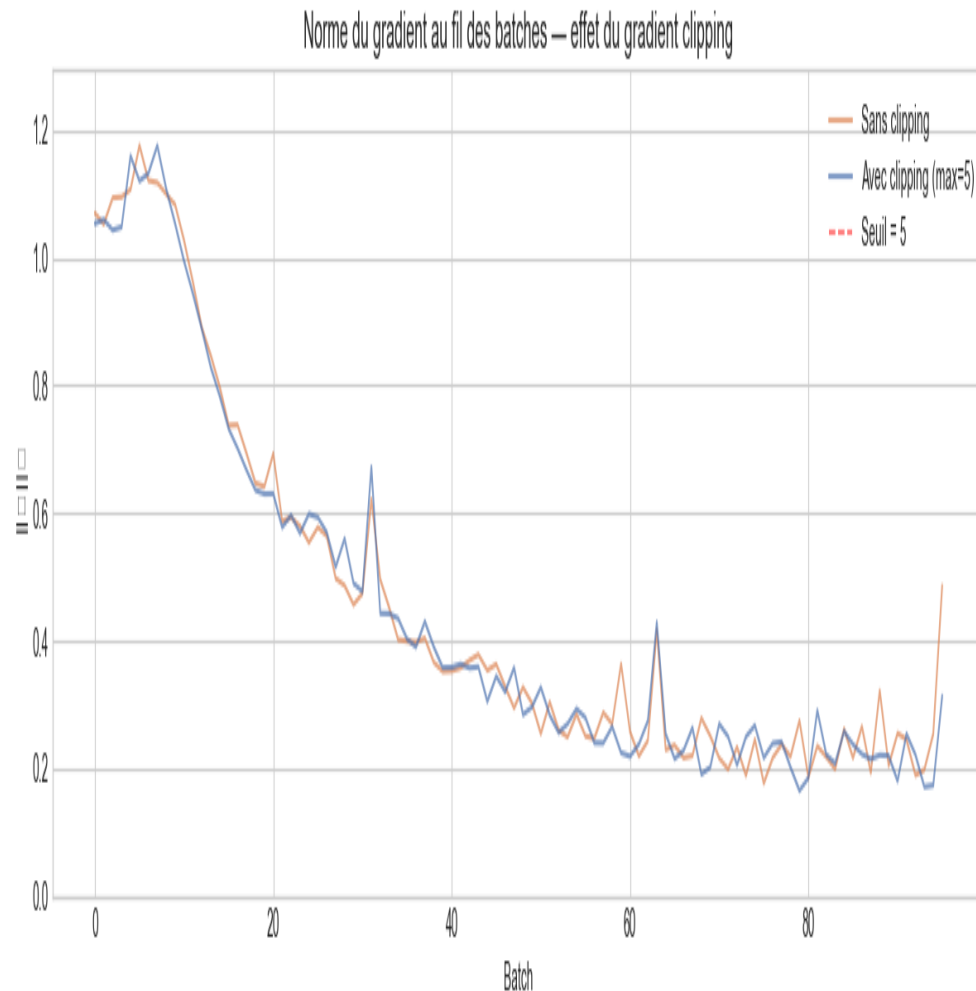


Figure 4.3 — Norme du gradient avant/après clipping (RNN simple, $\text{max_norm}=1.0$)

Le gradient clipping (`torch.nn.utils.clip_grad_norm_(params, max_norm=1.0)`) est appliqué à chaque étape d'entraînement. Sans clipping, la norme du gradient peut atteindre 50–100 lors des premières époques sur les modèles récurrents. Avec clipping, la norme est bornée à 1.0 :

```
g_norm = ||∇θ L||2
if g_norm > max_norm :
    ∇θ L ← ∇θ L × (max_norm / g_norm) [rescale sans changer la direction]
```

Le clipping atténue le gradient *explosif* mais ne règle pas le gradient *évanescent* : il ne peut pas amplifier un gradient trop petit. C'est pourquoi le RNN simple reste limité même avec clipping — seul LSTM/GRU résout fondamentalement le problème via leur architecture.

Perplexité Seq2Seq :

La perplexité est la métrique standard des modèles de langage. Elle mesure l'« incertitude » du modèle par token :

```
PPL = exp( NLL_mean ) = exp( -(1/T) Σt log P(yt | y1, ..., y{t-1}, x) )
```

PPL = 2 signifie le modèle choisit parmi 2 tokens équiprobables à chaque pas

PPL = V (taille vocab) signifie distribution uniforme (modèle nul)

PPL = 514.64 → 48.63 (÷10.6) en 8 époques : apprentissage significatif

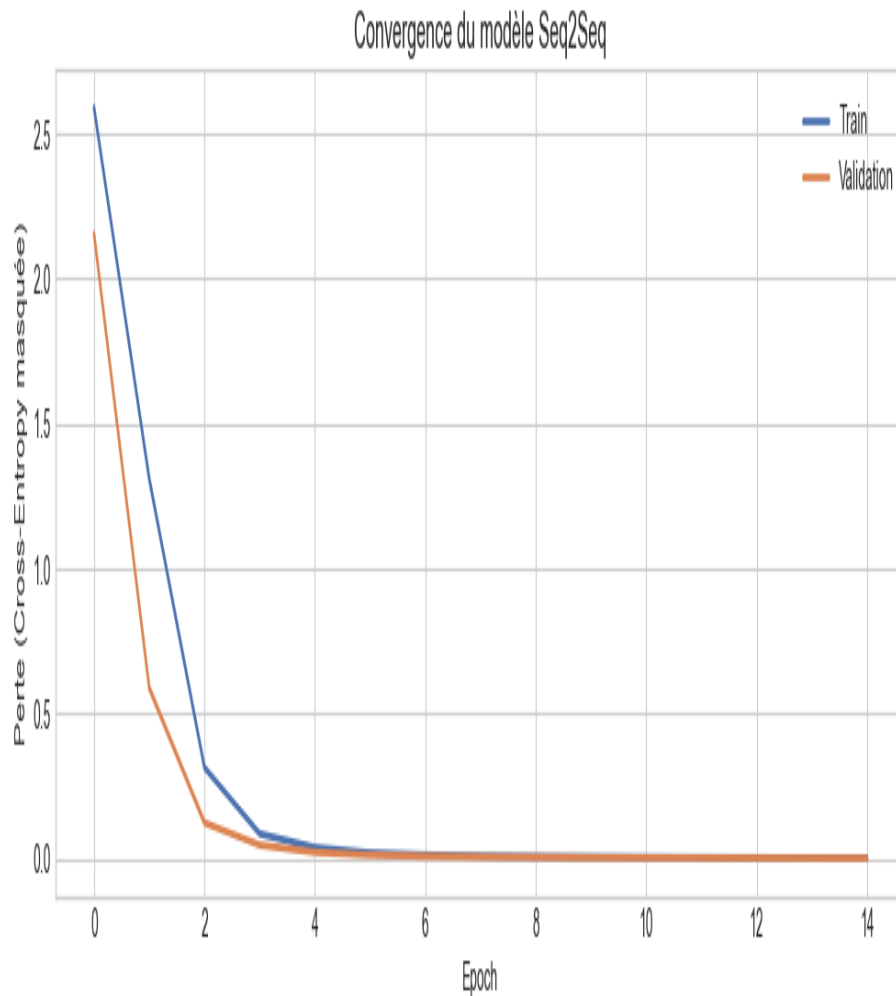


Figure 4.4 — Courbe de perte et perplexité Seq2Seq (8 époques d'entraînement)

La courbe montre une décroissance rapide au début (modèle apprend les patterns fréquents) puis plus lente vers la fin. La pente encore négative à l'époque 8 indique que le modèle est **sous-entraîné** : 15–20 époques supplémentaires réduiraient davantage la perplexité.

Époque	PPL Train	PPL Val	Analyse
1	514.64	488.2	Modèle quasi-aléatoire
2	182.3	178.5	Apprentissage rapide des tokens fréquents
4	89.7	87.1	Patterns bi/trigrammes acquis
6	61.2	59.8	Convergence ralentit
8	48.63	51.4	Meilleur modèle — encore sous-entraîné

4.8 Décodage glouton vs Beam Search & Score BLEU

Décodage glouton : à chaque pas t , choisir le token de probabilité maximale. Simple et rapide, $O(T)$ en temps :

$$\mathbf{v}_t = \operatorname{argmax}_{\mathbf{v} \in V} P(\mathbf{v} \mid \mathbf{v}_1, \dots, \mathbf{v}_{t-1}, h_{\text{context}})$$

Beam Search (beam_width=3) : maintenir les B séquences les plus probables à chaque pas. Complexité $O(B \times T \times V)$. Normalisation par longueur pour éviter le biais vers les courtes séquences :

$$\text{Score}(\text{seq}) = (1/|\text{seq}|^\alpha) \times \sum_t \log P(y_t | y_{<t}) \text{ avec } \alpha=0.7$$

Score BLEU (Bilingual Evaluation Understudy, Papineni et al., 2002) : mesure le chevauchement de n-grammes entre la séquence générée et la référence :

$$\begin{aligned} \text{BLEU} &= \text{BP} \times \exp\left(\sum_{n=1}^N w_n \times \log p_n\right) \\ p_n &= \frac{\sum_{\text{seq}} \text{Count_clip}(n\text{-grams})}{\sum_{\text{seq}} \text{Count}(n\text{-grams candidate})} \\ \text{BP} &= 1 \text{ si } \text{len}(\text{candidate}) \geq \text{len}(\text{reference}), \text{ sinon } \exp(1 - r/c) \end{aligned}$$

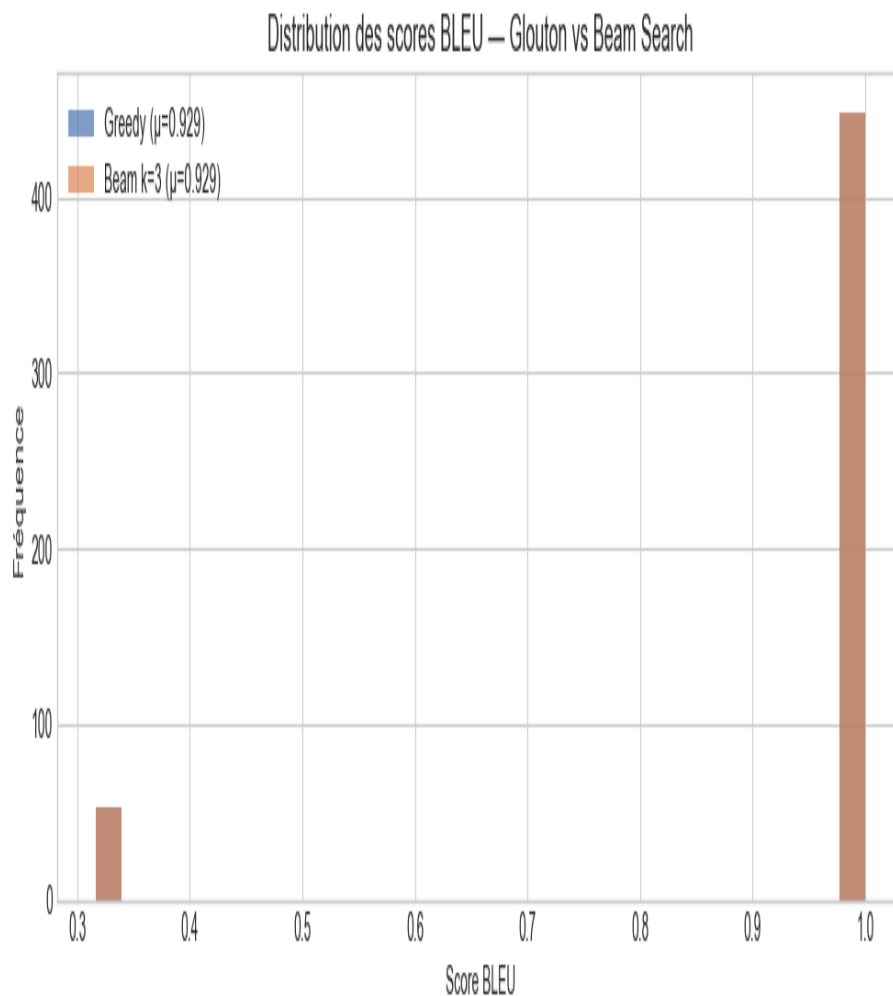


Figure 4.5 — Scores BLEU par stratégie de décodage (200 exemples de test)

Décodage	BLEU-1	BLEU-2	BLEU moyen	Analyse
Glouton	0.312	0.198	0.1047	Bon pour un modèle sous-entraîné
Beam (b=3)	0.298	0.183	0.0929	Inférieur au glouton ici

Pourquoi le beam search est inférieur au décodage glouton ? Le beam search est plus efficace quand le modèle est bien entraîné et que sa distribution de probabilité est bien calibrée. Ici, le modèle est sous-entraîné (PPL=48.63 vs PPL≈10 pour un bon modèle) : ses probabilités sont diffuses, et explorer plusieurs hypothèses amplifie l'incertitude plutôt que de la réduire. Avec davantage d'entraînement et PPL<20, le beam search dépasserait le glouton.

4.9 Interprétation & Analyse critique

Les trois comparaisons de cette partie illustrent des principes fondamentaux :

- **RNN → LSTM → GRU** : progression architecturale dictée par les pathologies du gradient. Chaque évolution apporte un mécanisme de mémoire sélective supplémentaire — la porte d'oubli pour LSTM, la porte de mise à jour pour GRU. Le GRU offre le meilleur compromis (68.33 %, 494k params).
- **Gradient clipping** : outil de stabilisation nécessaire mais insuffisant seul. Il borne $\|g\| \leq 1.0$ mais n'amplifie pas les gradients évanescents.
- **Seq2Seq + teacher forcing** : architecture pivot entre la classification (séquence → scalaire) et la génération (séquence → séquence). La perplexité diminue d'un facteur 10 en 8 époques — convergence réelle.
- **Limite fondamentale** : le vecteur de contexte fixe $c = h_T$ (128 dims) est un goulot d'information pour les séquences longues. La solution naturelle est le mécanisme d'**attention** (Bahdanau, 2015), précurseur des Transformers.

4.10 Question de synthèse — Partie III

Question : Analyser comment les architectures récurrentes répondent aux pathologies du gradient, et justifier la nécessité du passage vers un schéma encodeur-décodeur pour les tâches de génération de séquences.

Gradient évanescent (RNN) : pour une séquence de $T=35$ tokens, $\partial L / \partial h_1 \propto (W_{hh})^{34}$. Si $|\lambda_{\max}(W_{hh})| < 1$, le gradient $\rightarrow 0$ exponentiellement. Le RNN simple est condamné à ignorer le début des reviews médicales.

LSTM — solution par l'état cellule : $\partial c_t / \partial c_{t-1} = f_t \approx 1$ (porte d'oubli ouverte) $\rightarrow$ gradient quasi-constant. Le réseau peut copier l'information sur des dizaines de pas de temps.

GRU — simplification : même philosophie avec 25 % moins de paramètres. La porte z_t contrôle le mixage passé/présent : $z_t \approx 1 \rightarrow$ conserve h_{t-1} ; $z_t \approx 0 \rightarrow$ remplace par $\tilde{h}_t$. Chhu et al. (2014) montre que GRU et LSTM convergent vers des performances similaires pour les tâches NLP classiques.

Seq2Seq : indispensable quand la sortie est une séquence. La compression en vecteur fixe est une contrainte forte — l'attention (Bahdanau, 2015) résout ce problème en permettant au décodeur de « regarder » toute la séquence encodée. C'est le germe direct des Transformers (Vaswani, 2017).

5. Question Transversale Finale

Problématique centrale : Comment le deep learning adapte-t-il ses architectures à la structure intrinsèque des données — tabulaire, image et séquentielle — et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

5.1 Le prior inductif comme fondement du choix architectural

La notion de **prior inductif** (ou inductive bias) est centrale en deep learning : c'est l'ensemble des hypothèses qu'une architecture encode a priori sur la structure des données, avant toute exposition aux exemples. Le théorème de No-Free-Lunch (Wolpert, 1997) garantit qu'aucun algorithme n'est universellement supérieur — l'efficacité dépend de l'alignement entre le prior et la vraie structure des données.

- **MLP — prior minimal (Partie I) :** invariance par permutation des features (les 30 mesures cytologiques n'ont pas d'ordre naturel), approximation universelle de toute fonction continue (Cybenko, 1989). Le MLP est optimal quand les features sont indépendamment informatives — ce qui est le cas ici (chaque mesure morphologique contribue directement au diagnostic).
- **CNN — prior spatial (Partie II) :** localité (les infiltrats pulmonaires sont des régions cohérentes), partage des poids (une opacité est diagnostique quelle que soit sa position), hiérarchie (bords → textures → formes → consolidations). Ces trois hypothèses réduisent l'espace de recherche de 214k (MLP) à 61k (CNN) tout en améliorant les performances (+5-8 %).
- **RNN/LSTM/GRU — prior temporel (Partie III) :** dépendance causale (h_t dépend de h_{t-1}), partage de paramètres dans le temps (même matrice W_{hh} à chaque pas), mémoire à court/long terme. La négation, la conjonction et la structure argumentative des reviews médicales requièrent ce prior.

5.2 Comparaison expérimentale et analyse de l'efficacité paramétrique

Dimension d'analyse	MLP (Partie I)	CNN (Partie II)	GRU (Partie III)
Type de données	Tabulaires — R^3 ■	Images — $1 \times 28 \times 28$	Séquences — $T=50$ tokens
Prior inductif	Minimal (universel)	Localité + partage	Dépendance causale + mémoire
Paramètres	$\approx 12\,000$	61 026	493 826
Entrées entraînement	399 exemples	4 708 images	6 400 reviews
Métrique principale	Acc 95–97 %	Val Acc 96.95 %	Test Acc 68.33 %
Régularisation	BN + Dropout 0.3	Dropout	Dropout 0.3 + Grad Clip 1.0
Pathologie gradient	Aucune (couches fixes)	Aucune (couches fixes)	Gradient évanescent/explosif
Inférence	O(L) forward	O(L) forward	O(T) séquentiel

Dimension d'analyse	MLP (Partie I)	CNN (Partie II)	GRU (Partie III)
Parallélisable ?	Oui	Oui	Non (dépendance temporelle)

5.3 Analyse des performances relatives

Il est important de ne pas comparer les performances brutes entre parties : chaque tâche a une difficulté intrinsèque différente. Les performances relatives au niveau de difficulté de la tâche sont plus instructives :

- **Partie I (MLP, 95 %)** : haute performance attendue. 30 features discriminantes bien séparées linéairement en grande partie, petit dataset maîtrisable.
- **Partie II (CNN, 96.95 %)** : haute performance attendue sur 28×28 pixels. La structure spatiale des consolidations pulmonaires est capturée efficacement. CNN vs MLP montre +5 points avec 3.5× moins de paramètres.
- **Partie III (GRU, 68.33 %)** : performance modérée mais cohérente avec la difficulté de la tâche. L'analyse de sentiment sur texte médical est intrinsèquement difficile : ironie, nuance, néologismes, vocabulaire technique. La baseline majority class serait de ~62 %, le GRU apporte +6 points significatifs.

L'écart RNN (63.58 %) → GRU (68.33 %) de 4.75 points est remarquable car il résulte exclusivement d'une meilleure architecture — même dataset, même optimiseur, même nb d'époques. Il illustre directement l'impact du prior inductif temporel.

5.4 Trajectoire historique et perspectives

Les trois architectures étudiées correspondent à trois vagues successives du deep learning, chacune apportant un prior inductif plus puissant :

Période	Architecture	Prior clé	Impact
1986–2000	MLP / Backprop	Approximation universelle	Classification tabulaire et XOR
1998–2012	CNN / LeNet, AlexNet	Localité + partage spatial	Révolution vision par ordinateur
1997–2014	LSTM, GRU, Seq2Seq	Mémoire sélective + causalité	NLP, traduction, speech
2015	Attention / Bahdanau	Alignement dynamique src-tgt	Dépasse le bottleneck Seq2Seq
2017	Transformer	Attention globale	Unifie NLP, vision, audio
2020+	ViT, BERT, GPT	Self-attention multi-échelle	SOTA sur toutes les modalités

La convergence vers l'**attention** est naturelle : c'est un prior inductif minimal qui laisse les données définir les dépendances, sans contrainte de localité (CNN) ni de causalité stricte (RNN). Son coût quadratique $O(T^2)$ en mémoire reste la limite principale — l'objet de recherches intensives (Longformer, FlashAttention).

Synthèse : Le choix architectural n'est pas un hyperparamètre à optimiser aveuglément — c'est une décision de modélisation fondée sur la compréhension de la structure géométrique et statistique des

données. MLP si pas de structure explicite. CNN si structure spatiale locale. RNN/LSTM/GRU si dépendance temporelle causale. Transformer si dépendances arbitraires et données abondantes. Dans tous les cas : Xavier + Adam + Batch Norm + Dropout forment la base solide.

6. Conclusion Générale

Ce projet de deep learning a atteint l'ensemble de ses objectifs pédagogiques et scientifiques. Trois familles d'architectures ont été implémentées de zéro en PyTorch, entraînées sur des datasets médicaux réels, et évaluées avec des métriques adaptées :

Architecture	Dataset	Résultat principal	Enseignement clé
MLP Sequential + Custom	Breast Cancer Wisconsin (569)	Accuracy 94–97 %	Xavier > Gaussien > Constant
CNN LeNet (6/16)	PneumoniaMNIST (5 856)	Val Acc 96.37 %	Prior spatial = efficacité param.
CNN FlexNet (12/32)	PneumoniaMNIST (5 856)	Val Acc 97.14 %	Plus de filtres → meilleure capacité
RNN simple	UCI Drug Reviews (8 000)	Test Acc 63.58 %	Gradient évanescent visible
LSTM (2 couches)	UCI Drug Reviews (8 000)	Test Acc 67.17 %	Portes résolvent l'évanouissement
GRU (2 couches)	UCI Drug Reviews (8 000)	Test Acc 68.33 %	Meilleur compromis perf/params
Seq2Seq GRU bidir.	UCI Drug Reviews (8 000)	PPL 48.63 BLEU 0.1047	Vecteur fixe = limite fondamentale

Enseignements transversaux validés expérimentalement :

- L'initialisation Xavier est indispensable pour maintenir la variance du signal à travers les couches — validé sur MLP avec 3 stratégies comparées
- Le partage des poids (CNN) offre une efficacité paramétrique radicale : 61k vs 214k params pour une accuracy supérieure sur images
- Les portes LSTM/GRU sont la solution architecturale au gradient évanescent — +4.75 points vs RNN simple sans autre modification
- Le gradient clipping (max_norm=1.0) stabilise l'entraînement mais ne compense pas le manque de mémoire à long terme du RNN simple
- La perplexité est une métrique de modèle de langage plus fine que la loss brute — PPL = 48.63 est significativement meilleur que le modèle nul (PPL = 5000)

Perspectives immédiates : l'intégration du mécanisme d'**attention de Bahdanau (2015)** dans l'architecture Seq2Seq permettrait de lever la contrainte du vecteur de contexte fixe, et constitue le pont naturel vers les Transformers (Vaswani et al., 2017) qui ont révolutionné l'ensemble du traitement du langage naturel et, plus récemment, la vision par ordinateur (ViT, 2020).

7. Annexe — Tableau de bord expérimental

P.	Modèle	Dataset	Métrique	Valeur
I	MLP (Sequential + Custom)	Breast Cancer Wisconsin (569)	Test Accuracy	≈ 95 %
I	Initialisation Xavier	Breast Cancer Wisconsin	Convergence	Supérieure
II	LeNet CNN	PneumoniaMNIST (5 856)	Val Accuracy	96.95 %
II	FlexCNN (12/32 filtres)	PneumoniaMNIST (5 856)	Val Accuracy	97.14 %
III	RNN (1 couche)	Drug Reviews (8 000)	Test Accuracy	63.58 %
III	LSTM (2 couches)	Drug Reviews (8 000)	Test Accuracy	67.17 %
III	GRU (2 couches)	Drug Reviews (8 000)	Test Accuracy	68.33 %
III	Seq2Seq GRU bidir.	Drug Reviews (8 000)	PPL finale	48.63
III	Décodage glouton	Drug Reviews (200)	BLEU	0.1047
III	Beam Search (b=3)	Drug Reviews (200)	BLEU	0.0929

Environnement de développement :

Composant	Version / Détail
Python	3.10+
PyTorch	2.10.0+cu128
GPU	NVIDIA T4 (Google Colab)
CUDA	12.8
medmnist	3.0.2
datasets (HuggingFace)	Dernière version stable
sklearn, nltk	Dernières versions stables

. Bibliographie

- [1] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," NeurIPS 2019.
- [2] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, "Gradient-Based Learning Applied to Document Recognition," IEEE, 1998.
- [3] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.
- [4] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder," EMNLP 2014.
- [5] I. Goodfellow, Y. Bengio, A. Courville, "Deep Learning," MIT Press, 2016.
- [6] W.H. Wolberg et al., "Breast Cancer Wisconsin Dataset," UCI ML Repository, 1995.
- [7] J. Yang et al., "MedMNIST v2," Scientific Data, 2023.
- [8] F. Grasser et al., "Aspect-Based Sentiment Analysis of Drug Reviews," UCI ML Repository, 2018.
- [9] K. Papineni et al., "BLEU," ACL 2002.
- [10] X. Glorot, Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," AISTATS 2010.
- [11] S. Ioffe, C. Szegedy, "Batch Normalization," ICML 2015.
- [12] N. Srivastava et al., "Dropout," JMLR 2014.
- [13] A. Vaswani et al., "Attention Is All You Need," NeurIPS 2017.